

CMPT 120 — Strings, Lists, Files & Recommendations

Agenda

- Housekeeping
- String splitting & advanced operations
- Refreshers: list & string indexing. Comparison. We'll need both!
- File reading methods in Python
- Recommendation systems
- Practice review questions
- Live coding activity (scaffold)

Housekeeping

String Splitting & Advanced Operations

The `split()` Method

- `str.split()` breaks a string into a list based on a delimiter.
- Default delimiter = whitespace.

```
"apples bananas cherries".split()  
# ['apples', 'bananas', 'cherries']
```

Custom Delimiters

```
data = "2025-09-29"  
data.split("-")  
# ['2025', '09', '29']
```

The `join()` Method

```
words = ["apples", "bananas", "cherries"]  
", ".join(words)  
# 'apples, bananas, cherries'
```

Advanced String Operations

- **Slicing:** `s[start:end:step]`
- **Reversing:** `s[::-1]`
- **Membership:** `'cat' in 'concatenate'`
- **Case changes:** `.lower()`, `.upper()`, `.title()` (not on test)
- **Stripping:** `.strip()`, `.lstrip()` (not on test), `.rstrip()` (not on test)

Indexing Refresher

Indexing Basics

```
s = "HELLO"  
s[0] # 'H'  
s[-1] # 'O'
```

Wackier Slices

```
s = "PYTHON"  
s[0:4]    # 'PYTH'  
s[::2]    # 'PTO'  
s[::-1]   # 'NOHTYP'
```

Nested Indexing

```
words = ["CMPT", "Python", ["Wacky", "Indexing"]]  
words[2][1]  
# 'Indexing'
```

Comparisons & Logical Operators (refresher)

Comparison Operators

- `==` equal
- `!=` not equal
- `<`, `<=`, `>`, `>=`

Logical Operators

- `and` : both conditions true (strict)
- `or` : at least one true (not strict)
- `not` : negates condition ("flip" True to False)

New: File Reading in Python

Opening Files

```
with open("data.txt", "r") as f:  
    text = f.read()
```

`.read()` vs `.readline()` vs `.readlines()` vs "iterate over on `open` object"

- `f.read()` : whole file as a single string.
- `f.readline()` : one line at a time.
- `f.readlines()` : list of all lines.

Iterating Over a File

```
with open("data.txt", "r") as f:  
    for line in f:  
        print(line.strip())
```

Recommendation Systems

Basic Recommendation

- **Most Popular:** Suggest the most common item.
- Examples: most read news story, most bought product.

Advanced Recommendation

- **People Like You Also Liked:**
 - Andrea likes apples, bananas, cherries.
 - Bob likes durian, bananas, cherries.
 - Suggest Andrea try durian; Bob try apples.

Similarity Scores

- Compare users by overlap in preferences.
- Algorithm:
 - i. Initialize top score = 0, top person = "".
 - ii. For each person:
 - Calculate similarity with you.
 - If higher than top score → update.
 - iii. At end, top person = most similar.

6. Practice Review Questions

Indexing

```
singers = ["elsa", "anna", "snowman"]  
print(singers[2])    # ?
```

Negative Indexing

```
favourites = ["roti", "pita", "sourdough", "tortilla"]  
print(favourites[-2]) # ?
```

Sublist Slices

```
letters = ['a', 'b', 'c', 'd', 'e', 'f']  
print(letters[1:3]) # ?  
print(letters[:4]) # ?  
print(letters[3:]) # ?  
print(letters[-1]) # ?
```

Nested Loops

What does it mean to have a **nested loop**?

- Write a quick example.

Live Coding Scaffold

Activity Prompt

- Open fruit preference data
- Read its contents.
- Split text up into columns
- ...
- Make fruit recommendations to Alice!

Python Dictionaries

Dictionaries — Basic

- A collection of key: value pairs.
- Keys must be unique and hashable (immutable types like str, int, tuple).
- Fast lookups by key: “given a key, get a value”.

Create a Dictionary

```
student_ages = {  
    "Alice": 20,  
    "Bob": 19,  
    "Charlie": 21,  
}
```

Access by Key

```
student_ages["Alice"]      # 20  
student_ages.get("Alice")  # 20  
student_ages.get("Dana")   # None (safe: no KeyError)  
student_ages.get("Dana", 0) # 0 (default)
```

Add / Update / Remove

```
student_ages["Dana"] = 18      # add
student_ages["Alice"] = 21    # update
del student_ages["Bob"]       # remove (KeyError if missing)
student_ages.pop("Charlie", 0) # remove, return value or default
```

Membership & Views

```
"Alice" in student_ages      # True (checks keys)
20 in student_ages.values()  # True
list(student_ages.keys())    # ["Alice", "Dana", ...]
list(student_ages.items())   # [("Alice", 21), ("Dana", 18), ...]
```

Iterating

```
for name in student_ages:           # keys
    print(name, student_ages[name])

for name, age in student_ages.items(): # key, value pairs
    print(name, age)
```

Dictionaries — Advanced

- Nested structures for richer data.
- Counting with `get()` pattern.
- Dict comprehensions for compact builds.
- Merging and copying; mutability & order notes.

Nested Dictionaries

```
students = {  
    "Alice": {"age": 21, "major": "CS"},  
    "Dana": {"age": 18, "major": "Math"},  
}  
students["Alice"]["major"] # 'CS'
```

Counting with Dictionaries

```
fruits = ["apple", "banana", "apple", "cherry", "banana", "apple"]
counts = {}
for f in fruits:
    counts[f] = counts.get(f, 0) + 1
# {'apple': 3, 'banana': 2, 'cherry': 1}
```

Dict Comprehensions (NOT REQUIRED)

```
squares = {n: n*n for n in range(6)}  
# {0: 0, 1: 1, 2: 4, 3: 9, 4: 16, 5: 25}  
  
evens_squared = {n: n*n for n in range(10) if n % 2 == 0}
```

Merge, Copy, and Order

```
a = {"x": 1, "y": 2}
b = {"y": 99, "z": 3}

merged = a | b          # Python 3.9+: {'x': 1, 'y': 99, 'z': 3}
a.update(b)            # a becomes {'x': 1, 'y': 99, 'z': 3}

alias = a               # same object (changes reflect in both)
shallow_copy = a.copy() # new dict (top-level only)
```

- Dicts preserve insertion order (Python 3.7+), but are not automatically sorted.

Keys Must Be Hashable

```
ok = { (1, 2): "point", "name": "Alice" }  
bad = { [1, 2]: "list" }    # TypeError: unhashable type: 'list'
```

References

- [Runestone Academy: Working with Data Files](#)
- [Runestone Academy: Lists and Strings](#)
- [StackOverflow: Difference between read, readline, readlines](#)
- [Runestone Academy: Logical Operators](#)